

Mock autotests — setup and run

This section describes how to bring up a local OmegaClaw container running with the deterministic LLM mock and run the `test_*_mock.py` suite against it.

For the live-provider counterpart (real LLM, grading scheme, parameters table) see `Autotests/README_live.pdf`.

1. Prerequisites

- Docker engine on the host.
- Repository checked out, working from its root.
- Python virtual environment under `Autotests/venv` with `pytest` installed.

2. Build the local image

The mock infrastructure is part of the source tree, so the image must be built locally rather than pulled from the registry.

```
docker build -t omegaclaw:mock .
```

3. Start the container with the Test provider

The container connects back to the host on TCP port 9765 to reach the mock controller. `TEST_API_KEY` must hold the host IP that is reachable from inside the container — under the default Docker bridge this is `172.17.0.1`.

```
docker run -d -it \  
  --name omegaclaw_mock \  
  --user 65534:65534 \  
  --init \  
  --network bridge \  
  --security-opt no-new-privileges:true \  
  --volume omegaclaw-mock-memory:/PeTTa/repos/OmegaClaw-Core/memory/ \  
  --tmpfs /tmp:size=64m,mode=1777,exec \  
  --tmpfs /run:size=16m,mode=755 \  
  --tmpfs /var/tmp:size=64m,mode=1777,exec \  
  -e TEST_API_KEY=172.17.0.1 \  
  -e OMEGACLAW_AUTH_SECRET=0000 \  
  omegaclaw:mock \  
  IRC_channel="#metaclaw_mock" \  
  provider="Test" \  
  embeddingprovider="Local"
```

Notes:

- `provider="Test"` selects the mock LLM dispatcher.
- `embeddingprovider="Local"` keeps the embedding model in-process (no network call).
- `TEST_API_KEY=172.17.0.1` is the host's docker-bridge address.

- OMEGACLAW_AUTH_SECRET=0000 matches the value the test harness sends as auth 0000.
- The IRC channel can be any unique string; pick one not used by another concurrent run.

Wait until the container has registered on IRC and joined the channel:

```
docker logs omegaclaw_mock 2>&1 | grep -E "Joining|Registered"
```

4. Configure the test environment

Export the variables the test harness reads. The values must match the container name and IRC channel chosen at step 3.

```
export OMEGACLAW_CONTAINER=omegacrawl_mock
export OMEGACLAW_IRC_CHANNEL="#metacrawl_mock"
export OMEGACLAW_GIT_TOKEN=<github_pat> # only required by test_git_p
```

5. Run the suite

```
cd Autotests
source venv/bin/activate
pytest -s -v mock/test*_mock.py
```

The mock controller is provided by a session-scoped fixture in `mock/conftest.py`, so it is started once per pytest session. Expected output: 26 passed.

6. Tear down

```
docker rm -f omegacrawl_mock
docker volume rm omegacrawl-mock-memory
```

Tests description

All 26 tests follow the same pattern: the test registers a fixed mock-LLM answer for the prompt via `llm.set_answer(prompt, response)`, delivers the prompt to the agent over IRC via `send_prompt(prompt)`, then verifies the resulting skill calls and side effects (filesystem, history, metta, ChromaDB, docker logs). Because the LLM is deterministic, no `try_with_clarification` retries are needed — every test either passes on the first attempt or fails outright.

Creating files

1. test_create_file_mock.py

Creates `/tmp/testcat/hello.txt` containing exactly Hello.

- Mock answer: `(shell "mkdir -p /tmp/testcat") (write-file "/tmp/testcat/hello.txt" "Hello")`.
- Checks: directory exists, file exists, permissions start with `-rw`, `mtime ≥ test start`, content is Hello (with or without trailing newline).

2. test_create_empty_file_mock.py

Creates an empty `/tmp/test_empty/hello.txt`.

- Mock answer: `(shell "mkdir -p /tmp/test_empty") (write-file "/tmp/test_empty/hello.txt" "")`.
- Checks: file is created, `cat` returns an empty string.

3. test_create_script_mock.py

Creates `/tmp/test_script/date.sh` — a shell script that prints the current date.

- Mock answer: `(shell "mkdir -p /tmp/test_script") (write-file "/tmp/test_script/date.sh" "#!/bin/bash\ndate\n") (shell "chmod +x /tmp/test_script/date.sh")`.
- Checks: file exists, is executable (x in permissions), harness runs it via `sh date.sh` and verifies the output contains the current year (UTC or local).

Editing files

4. test_edit_add_timestamp_mock.py

Appends the current timestamp as a new line to a pre-created `note.txt`.

- Mock answer: `(shell "date -Iseconds >> {TARGET_FILE}")`.
- Checks: mtime changed, last line of the file contains ≥ 4 digits.

5. test_edit_delete_line_mock.py

Deletes the second line from a pre-created 3-line file.

- Mock answer: `(shell "sed -i 2d {TARGET_FILE}")`.
- Checks: mtime changed, exactly 2 lines remain, lines 1 and 3 intact.

6. test_edit_append_line_mock.py

Appends a 4th line `Delta` to a pre-created 3-line file (`Alpha / Bravo / Charlie`).

- Mock answer: `(shell "printf '%s\\n' Delta >> {TARGET_FILE}")`.
- Checks: mtime changed, 4 lines total, first 3 unchanged, 4th line equals `Delta`.

7. test_convert_format_mock.py

Converts `document.md` $\rightarrow$ `document.txt`, preserving textual content.

- Mock answer: `(shell "cp {SOURCE_FILE} {DEST_FILE}")`.
- Checks: `.txt` exists, contains the keywords `My Title`, `Some paragraph text`, `item one`, `item two`.

Running shell scripts

8. test_run_error_script_mock.py

Runs a syntactically broken pre-created script and captures stdout + stderr to a file.

- Mock answer: `(shell "sh {SCRIPT_FILE} > {OUTPUT_FILE} 2>&1")`.
- Checks: `output.txt` exists, contains the literal `start` (stdout) AND at least one of `error` / `syntax` / `unexpected` / `missing` / `not found` (stderr); container is still alive.

9. test_run_repeated_mock.py

Runs `dateupdate.sh` exactly 10 times in a row.

- Mock answer: ten consecutive `(shell "{SCRIPT_FILE}")` calls (one per run).
- Checks: `update.txt` exists with `mtime ≥ start`, has $≥ 10$ lines, every line contains date-like digits.

Internet search

10. test_search_basic_mock.py

Sends prompt "What is SingularityNet? Search the web."

- Mock answer: `(send "SingularityNet (SNET) is a decentralized AI marketplace founded by Ben Goertzel. Its native token AGIX powers a network of AI agents and services, and it is part of the broader ASI Alliance ecosystem.")`.
- Checks: the reply contains at least one of `singularitynet`, `agi`, `blockchain`, `decentralized`, `marketplace`, `goertzel`.

11. test_search_weather_mock.py

Returns a synthetic Valencia weather reply (the live variant cross-checks against open-meteo; the mock variant fixes a synthetic reference `REF_TEMP_C = 18.0` and stays fully offline).

- Mock answer: `(send "Current weather in Valencia, Spain: about 18.0°C.")`.
- Checks: the reply contains a plausible Celsius number (range `[-20; 50]`); at least one of those numbers is within $±10$ °C of the synthetic reference `18.0 °C`.

12. test_search_invalid_mock.py

Asks about a gibberish string.

- Mock answer: `(send "No results found for <gibberish>. The string appears to be gibberish – no meaningful matches.")`.
- Checks: the reply contains a negation phrase (`no results`, `not found`, `gibberish`, `nonsense`, `no meaning`, `unknown`, ...).

13. test_tavily_search_mock.py

Live variant exercises the external Tavily uAgent. The mock variant cannot reach it deterministically, so the mocked response delivers the answer directly via `(send ...)` and the assertion narrows to "did the agent surface a real `Fetch.ai`-specific reply".

- **Mock answer:** (send "Fetch.ai (FET) is a decentralized AI blockchain platform powering autonomous economic agents (uAgents). Recent news covers the ASI Alliance roadmap, FET token activity, and integration work with SingularityNET and CUDOS."). The tavily-search skill itself is **not** invoked under the mock.
- **Checks:** a (send ...) exists whose body contains at least one strict Fetch keyword (fetch.ai, fetch ai, fet , asi alliance, humayun, uagent, decentralized, blockchain, token) AND none of the delivery-error markers (delivery failed, tavily-search failed, currently unavailable, ...).

14. test_technical_analysis_mock.py

Live variant exercises the external technical-analysis uAgent. The mock variant cannot reach it deterministically, so the mocked response delivers the TA summary directly via (send ...) and the assertion narrows to "did the agent surface TA-style content for the requested ticker".

- **Mock answer:** (send "AAPL (Apple) is showing bullish momentum: RSI is rising, MACD crossed above its signal line, and the 50-day SMA is above the 200-day. Composite indicators point to a buy signal with strong trend strength."). The technical-analysis skill itself is **not** invoked under the mock.
- **Checks:** a (send ...) exists whose body mentions the ticker (aapl or apple) AND at least one TA indicator (rsi, macd, sma, bullish, bearish, buy signal, trend, momentum, ...) AND none of the delivery-error markers.

Memory

15. test_memory_chromadb_mock.py

Requests the agent to remember a fact tagged with marker CI-SMOKE-<run_id>.

- **Mock answer:** (remember "Unique smoke marker CI-SMOKE-<run_id> was emitted by CI.").
- **Checks:** (remember ...) was invoked with the marker; vector count in the embeddings table of chroma.sqlite3 grew by ≥ 1 .

16. test_memory_history_mock.py

Sends "Acknowledge with one short line that you received marker <run_id>." and verifies the entry in history.metta.

- **Mock answer:** (send "Acknowledged marker <run_id>.").
- **Checks:** an s-exp record referencing REQ-<run_id> appears in history; the agent issued (send ...); file mtime and size grew.

17. test_skill_metta_mock.py

Asks the agent to evaluate a short MeTTa expression and report the result.

- **Mock answer:** (metta "(+ 2 2)") (send "The metta skill evaluated (+ 2 2) and returned 4.").
- **Checks:** (metta ...) was invoked; the agent then issued a (send ...). Semantic

correctness of the MeTTa expression is not checked — the goal is to exercise the skill.

18. test_skill_pin_mock.py

Gives a multi-step task ("restarting servers alpha → beta → gamma, just finished alpha") and expects the agent to track progress with `pin`.

- Mock answer: `(pin "Server restart progress: alpha done; beta and gamma pending.") (send "Tracking: alpha done, beta and gamma pending.")`.
- Checks: `(pin ...)` was invoked whose argument references either the `run_id` or one of the keywords `step / alpha / beta / gamma / restart / server / done`; the agent acknowledged via `(send ...)`.

Working with git

19. test_git_pull_public_mock.py

Agent clones a public repository over anonymous HTTPS, no token.

- Mock answer: `(shell "rm -rf {TARGET_DIR} && git clone {remote} {TARGET_DIR}")`.
- Checks: `.git/` appears, HEAD points to a real commit, ≥ 1 tracked file in HEAD, origin matches the expected remote URL (normalized — trailing / and `.git` ignored).

20. test_git_local_commit_mock.py

Agent runs `git init`, `git add`, `git commit` locally inside the container.

- Mock answer: chain of `(shell "git -C {TARGET_DIR} init") (shell "...write file...") (shell "git -C {TARGET_DIR} add -A") (shell "git -C {TARGET_DIR} commit -m 'add hello <run_id>')"`.
- Checks: HEAD has at least one commit, commit subject contains the `run_id` (warning, not failure), the file is present in the tree.

21. test_git_push_to_remote_mock.py

Agent clones a remote, creates branch `qa/run-<id>`, adds a file, commits, and pushes.

- Mock answer: single `(shell "rm -rf ... && git clone ... && cd ... && git checkout -b ... && printf ... > <file> && git add -A && git commit -m '...' && git push -u origin ...")`.
- Parameters via env vars: `OMEGACLAW_GIT_TOKEN` (token; never appears in code) and `OMEGACLAW_GIT_REMOTE` (default `https://github.com/OmegaSing/Test-Repopo`). Test is skipped if the token variable is unset.
- Checks: branch present on remote (GitHub API 200), file present on branch, the shell call included `git push`, credentials wiped on teardown.

Multi-skill tests

22. test_run_create_dirs_mock.py

Agent writes `mkdirs.sh` and runs it. The script must create `test1`, `test2`, `test3` inside `/tmp/test_dirs/`.

- Mock answer: `(write-file "{SCRIPT_PATH}" "#!/bin/bash\nmkdir -p .../test1 .../test2 .../test3\n") (shell "chmod +x {SCRIPT_PATH}") (shell "{SCRIPT_PATH}")`.
- Checks: all three directories exist with fresh mtimes; agent invoked `(write-file ...)` referencing `mkdirs.sh`; agent invoked `(shell ...)` to run the script. Diagnostics print `wf=<count>`, `sh=<count>`, `perms=<...>` to make stalls obvious.

23. test_memory_episode_mock.py

Two-turn flow: tells the agent that the user's dog Barney lost a baby tooth, waits 5 seconds, then asks to recall when this happened.

- Turn 1 mock answer: `(remember "Barney the dog lost his first baby tooth at the vet today.")`.
- Turn 2 mock answer: `(query "Barney tooth") (send "Barney the dog lost his first baby tooth on <YYYY-MM-DD>. The milestone is recorded in my notes.")`.
- Checks (turn 1): `(remember ...)` was invoked whose argument contains `tooth` or `Barney`. Checks (turn 2): `(query ...)` or `(episodes ...)` was invoked; the reply contains at least one of `dog / tooth / lost`; the reply contains the captured seed date in `YYYY-MM-DD` format.

24. test_skill_query_mock.py

Two-turn flow: plant a unique color (`azure-<run_id>`) via `remember`, wait for embeddings to settle, then ask the agent to recall it via `query` (embedding lookup, not timestamp lookup).

- Turn 1 mock answer: `(remember "My favorite color is azure-<run_id>.") (send "Stored: favorite colour is azure-<run_id>.")`.
- Turn 2 mock answer: `(query "favorite color") (send "Your favorite color is azure-<run_id>.")`.
- Checks (turn 1): `(remember ...)` carried the secret color. Checks (turn 2): `(query ...)` was invoked; the reply mentions the secret color verbatim.

25. test_skill_episodes_mock.py

Two-turn flow: send a message tagged with a unique keyword (no `remember`), capture the timestamp, then ask the agent to use `episodes` (timestamp lookup, not `query`) to recall what was discussed at that earlier time.

- Turn 1 mock answer: `(send "Acknowledged keyword <marker>.")`.
- Turn 2 mock answer: `(episodes "<seed_ts>") (send "The unique keyword was <marker>.")`.
- Checks (turn 1): the turn is recorded in `history.metta` with a timestamp. Checks (turn 2): `(episodes ...)` was invoked for the seed timestamp; the reply mentions the original marker.

26. test_complex_weather_flow_mock.py

Four-step pipeline: search NY weather → write `w.txt` with the forecast → write `p.sh` extracting the first Celsius number into `t.txt` → run `p.sh`. Because the mock controls only the LLM dispatch (the network-bound `search` skill is not exercised), the mocked response provides the forecast text directly.

- **Mock answer:** `(write-file "/tmp/wflow/w.txt" "New York tomorrow: clear, high 22 degrees Celsius.") (write-file "/tmp/wflow/p.sh" "#!/bin/bash\ngrep -oE '[0-9]+' /tmp/wflow/w.txt | head -1 > /tmp/wflow/t.txt\n") (shell "chmod +x /tmp/wflow/p.sh") (shell "/tmp/wflow/p.sh")`.
- **Checks:** `w.txt` exists; history contains `(write-file ...)` referencing `w.txt`; `p.sh` exists with executable bit; history contains `(write-file ...)` or `(shell ...)` referencing `p.sh`; `t.txt` exists; history contains `(shell ...)` running `p.sh`; `t.txt` content is a number in the range `[-60; 120]`; content length ≤ 40 characters.